

Flow Shop Scheduling

Scheduling - Single Stage

Consider a set of jobs $\{1, 2, \dots, n\}$ which need to be processed by a single machine/stage/activity.

e.g. a supermarket queue at one checkout.



Makespan – Total time taken to process all jobs

Completion Time – Total time jobs are waiting in the queue and take to be processed.

e.g. You are buying a Kit-Kat and are behind someone with a full trolley.

If they are kind enough to let you through, what happens to the **Makespan** and the **Completion Time**?

Scenario 1 don't let pass
2 let pass

trolley
10

kitkat
ε

10
10
ε 20 + ε

ε
ε + 10 + ε
10

Makespan isn't concerned with order. The checkout takes the same amount regardless of order.

Completion Time however is vastly different. If you are not let by, you both wait for almost the entire **Makespan**.

Double

Shortest Processing Time

Consider a set of jobs $\{1, 2, \dots, n\}$ which need to be processed by a single machine (stage).
e.g. a supermarket queue at one checkout.

C_{max} - Makespan, i.e. the last completion time

C_j - Completion time of job j *in queue + job time*

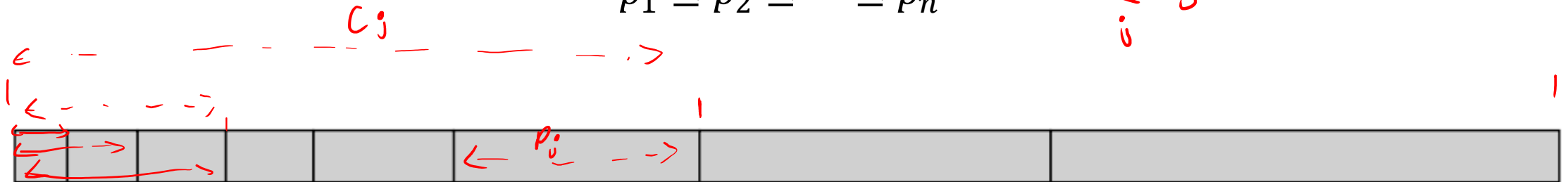
$\sum_j C_j$ - Sum of all completion times.

p_j - length of job j

Shortest processing time, order the jobs in ascending order

$$p_1 \leq p_2 \leq \dots \leq p_n$$

$$\sum_i C_j$$



$$\underline{a} \cdot \underline{b} = 5 \cdot 6 + 3 \cdot 7 + 2 \cdot 2 + 0 \cdot 5$$

$$\min(\underline{a} \cdot \underline{b})$$

Shortest Processing Time

$\underline{a} \cdot \underline{b}$ at minimum
be a

$$\rightarrow \underline{a} = (5, 3, 2, 0) \quad 4$$

style $5 \gg 3 \gg 2 \gg 0$

$$\underline{b} = (6, 7, 2, 5) \quad 4$$

$b_2 > b_3$

Lemma 1: Let $\underline{a}$ and vector of descending order and $\underline{b}$ a vector of similar length of no specific order. The minimum scalar product of $\underline{a} \cdot \underline{b}$ occurs when $\underline{b}$ is of ascending order.

Proof:

$$\underline{a} = (a_1, \dots, a_n), \quad \underline{b} = (b_1, \dots, b_n)$$

Such that:

$$a_1 \geq a_2 \geq \dots \geq a_n$$

$a_i \geq a_j$

Assume we have two indices $i < j$ (not necessarily adjacent) such that:

$$b_i > b_j$$

Define a new vector $\underline{b}'$, where these two elements are swapped.

$$\underline{a} = (\dots, a_i, \dots, a_j, \dots), \quad \underline{b} = (\dots, b_i, \dots, b_j, \dots), \quad \underline{b}' = (\dots, b_j, \dots, b_i, \dots)$$

a_i, b_i $b_j > b_i$ b_i, b_j

$3 \quad 6 \quad 3 \quad 6$

Shortest Processing Time

$$a_i b_j - a_i b_i = a_i (b_j - b_i)$$

$$a_j b_i - a_j b_j = a_j (b_i - b_j)$$

$$a \cdot b' = \dots + \dots + a_i b_j + \dots + a_j b_i + \dots$$

$$a \cdot b = \dots + \dots + a_i b_i + \dots + a_j b_j + \dots$$

$$a \cdot b' - a \cdot b = a_i (b_j - b_i) + a_j (b_i - b_j)$$

$$= (a_i - a_j)(b_j - b_i)$$

$$-a_j (b_j - b_i)$$

$$= a_j (b_i - b_j)$$

But $a_i \geq a_j \Rightarrow a_i - a_j \geq 0$ and $b_i > b_j \Rightarrow b_j < b_i \Rightarrow b_j - b_i < 0$

$$\therefore (a_i - a_j)(b_j - b_i) \leq 0$$

$\Rightarrow$

$$a \cdot b' - a \cdot b \leq 0$$

$\Rightarrow$

$$a \cdot b' \leq a \cdot b$$

decrease

Thus swapping any two elements in b where $b_i > b_j$ results in a equal or lower scalar product

Shortest Processing Time

Therefore for any b not in ascending order, there exists a swap which results in an equal or lower scalar product $a \cdot b$.

Thus the minimum is attained when b is in ascending order. ■ Q.E.D.
When no swaps left

a · b

a ascending

b descending

Shortest Processing Time

in order of (smallest jobs)

Theorem: SPT is optimal for minimising the sum of completion times $\sum_j C_j$

Proof:

$$C_1 = p_1, \quad C_2 = p_1 + p_2, \quad \sum_{j=1}^n C_j = \sum_{i=1}^n p_i$$

$$C_3 = p_1 + p_2 + p_3$$

$$C_3 = \sum_{i=1}^3 p_i$$

$$C_{max} = \sum_{j=1}^n C_j = p_1$$

$$\sum_{j=1}^n \sum_{i=1}^j p_i$$

$$+ p_1 + p_2 \quad C_2$$

$$+ p_1 + p_2 + p_3 \quad C_3$$

$$\vdots p_1 + p_2 + p_3 + p_4 \quad C_4$$

$$= np_1 + (n-1)p_2 + \dots + 2p_{n-1} + p_n \quad C_n$$

$$= (n, n-1, \dots, 2, 1) \cdot (p_1, p_2, \dots, p_{n-1}, p_n)$$

$$\underline{a} = (x, y)$$

$$\underline{b} = (a, b)$$

$$\underline{a} \cdot \underline{b} = xa + yb$$

$$(x, y) \cdot (a, b)$$

$(n, n-1, \dots, 2, 1)$ is of descending order, therefore to obtain a minimum by the previous lemma $(p_1, p_2, \dots, p_{n-1}, p_n)$ needs to be of ascending order, i.e. shortest processing time first.

$$(n, n-1, \dots, 2, 1) \cdot (p_1, p_2, \dots, p_n)$$



Job Shop Scheduling

Hard

This involves scheduling multiple jobs on multiple machines.

$J_1 \rightarrow J_2 \rightarrow J_3$
 $J_2 \rightarrow J_1 \rightarrow J_3$

e.g.

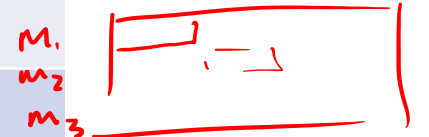
3 machines

Product 1

2

3

Jobs	Sequence of Machines	Process Time on Machines
$\rightarrow J_1$	M_1, M_2, M_3	5,3,8
J_2	M_1, M_3	4,6
J_3	M_3, M_2	2,7



This is known to a class of problems called NP ←

$NP \neq P$

Thought to be impossible to solve in polynomial-time. i.e. as problem increases, the time to solve evolves quicker than a polynomial.

Look for a 'good' solution! Optimal

You will see ways of finding a 'good' solution to this in the 3rd year

Trivia Time...



The average human body contains how many pints of blood?

- 9

The Statue of Liberty was given to the US by which country?

- France

What title is shared by a 1983 Lionel Richie song and a 2015 Adele song?

- Hello

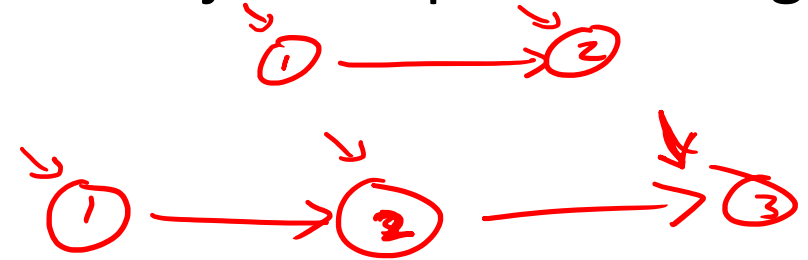
Flow Shop Scheduling

Permutation

Computers Processors

We will deal with a simpler set up than the general job shop scheduling problem.

- note we will use activity-on-node for this



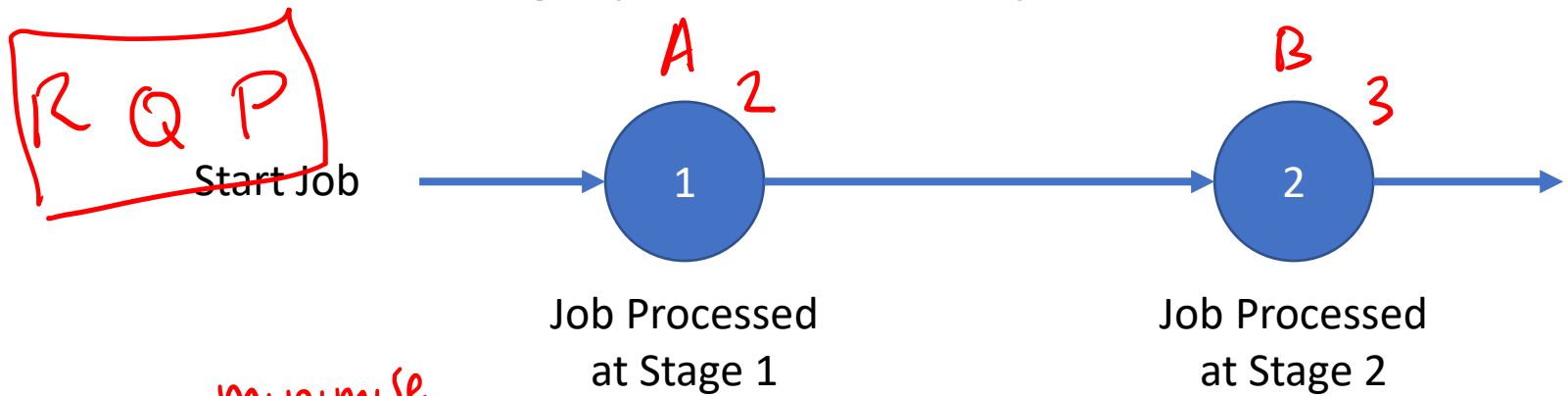
Often a company will have multiple jobs (e.g. products) that go through multiple stages (machines) sequentially.

Each job may require a different amount of time at each stage.

Example

n Jobs
 $\frac{n!}{m!}$
NP

Consider a 2 stage process in sequence.



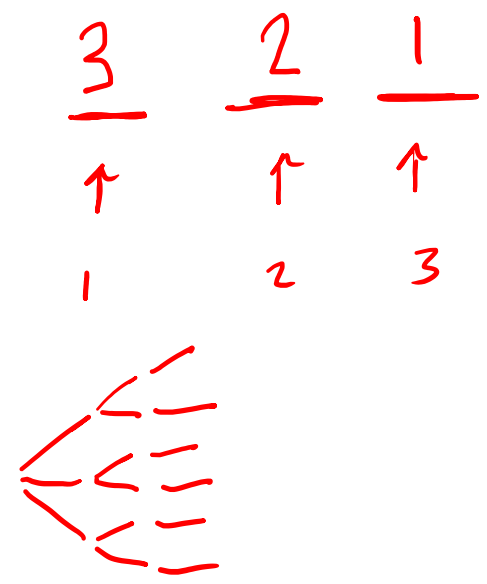
1 2 3
P Q R
P R Q
P R P Q
R Q P
Q R P
Q P R

minimise
time

Each job requires different times at each stage (activity).

e.g.

		P	Q	R
Machine	A	2	4	6
	B	3	3	7



Example

Jobs		P	Q	R
Machine	A	2	4	6
	B	3	3	7

Schedule jobs in order they appear. PQR

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Machine	A	✓	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗							
	B			✗	✗	✗		✗	✗	✗				✗	✗	✗	✗	✗	✗	✗

	P
	Q
	R

↑ idle

2 possibilities
makespan out of 6

n!

Is this optimal for Makespan? i.e. is everything completed as quickly as possible?

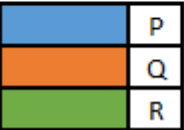
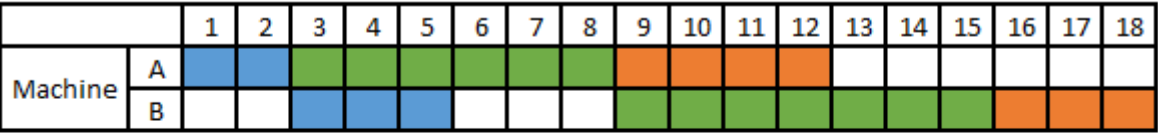
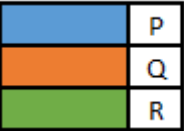
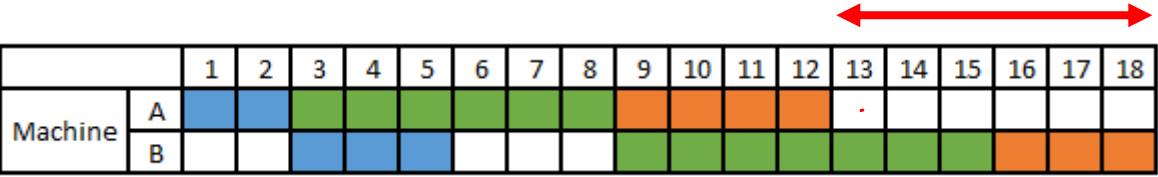
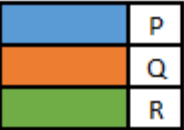
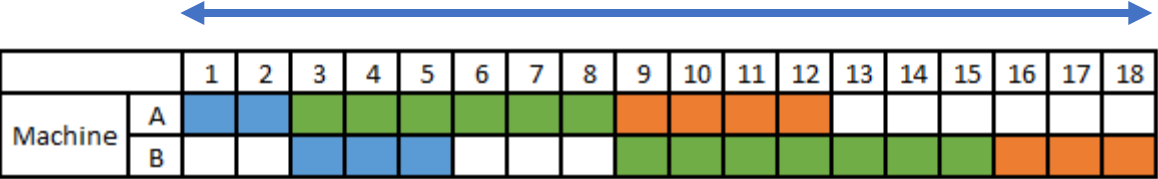
No PRQ

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Machine	A									✗	✗	✗	✗						
	B									✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

makespan

	P
	Q
	R

Makespan and Idle Time



Makespan is the total time required to completed all jobs

Idle Time is the time a stage (machine) is idle.

Stage A is idle for 6 units of time.

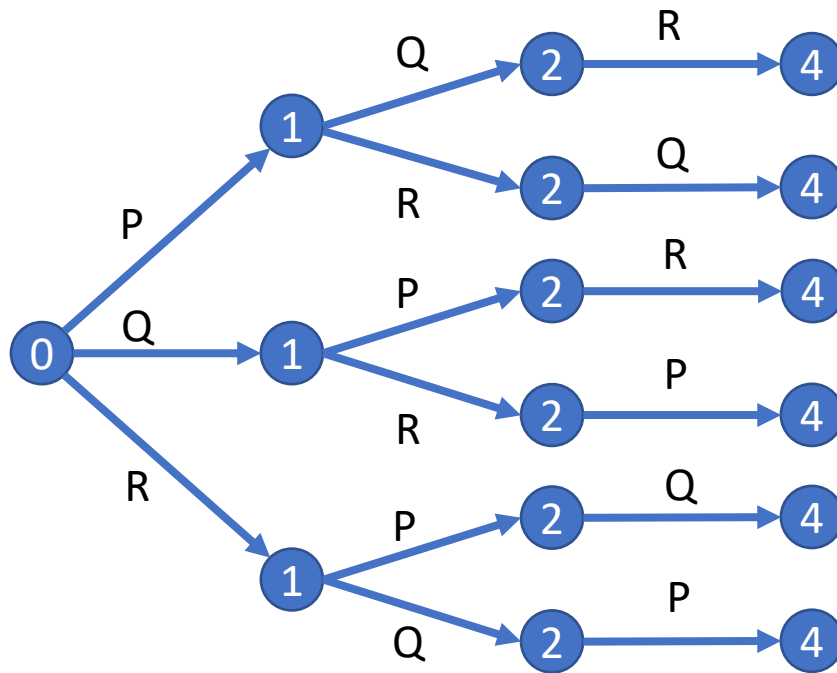
1)

Stage B is idle for 2+3=5 units of time.

Optimality

Optimality is difficult to check!

How many possible permutations. No repetitions and order matters!



$$3 \times 2 \times 1 = 6$$

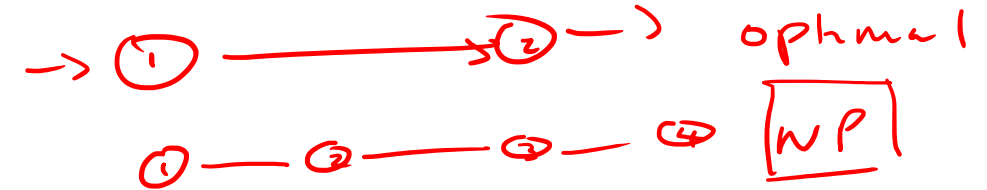
OR

$$3! = 6$$

For n jobs there are $n!$ ways of scheduling.

e.g. 10 jobs, $10! = 3628800$

Johnson's Rule



- Minimises Makespan optimal
- Optimal solution for 2 stages (machines) ←
- Optimal solution for 3 stages (machines) given additional conditions
 - Not covering this...
- Everything else is in NP
- In general problems like these require clever guessing (heuristics)

It effectively schedules the jobs which are small for the first stage at the beginning, and those that are small for the second stage at the end.

<https://www.rand.org/content/dam/rand/pubs/papers/2008/P402.pdf>

Johnson's Rule (Two stage)

- Steps

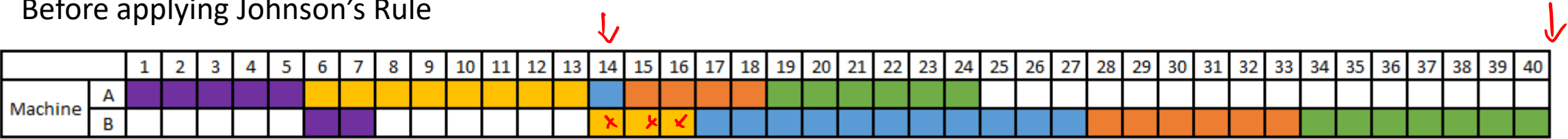
1. List the jobs ✓
2. Select the job with the shortest activity time, break ties arbitrarily
 - If shortest is for first stage, schedule as early as possible
 - Else if shortest is for second stage, schedule as late as possible
3. Remove the job from list. ✓
4. Repeat 2 and 3 until no jobs are left to schedule

Example

Jobs		P	Q	R	S	T
Machine	A	5	8	1	4	6
	B	2	3	11	6	7

Schedule					
----------	--	--	--	--	--

Before applying Johnson's Rule



	R
	S
	T
	Q
	P

Metric	Time
Makespan	40
Idle Time of A	16
Idle Time of B	11
Total Idle Time	27

Example

1. List the jobs
2. Select the job with the shortest activity time.
 - If shortest is for first activity, schedule as early as possible
 - Else if shortest is for second activity, schedule as late as possible
3. Remove the job from list.
4. Repeat 2 and 3 until no jobs are left to schedule

Jobs		P	Q	R	S	T
Machine	A	5	8	1	4	6
	B	2	3	11	6	7

Schedule					
----------	--	--	--	--	--



Jobs		P	Q		S	T
Machine	A	5	8		4	6
	B	2	3		6	7

Schedule	R				
----------	---	--	--	--	--

Example

makespan is optimal

Jobs		P	Q	R	S	T
Machine	A	5	8	1	4	6
	B	2	3	11	6	7

Schedule					
----------	--	--	--	--	--

Jobs		P	Q		S	T
Machine	A	5	8		4	6
	B	2	3		6	7

Schedule	R				
----------	---	--	--	--	--

Jobs			Q		S	T
Machine	A		8		4	6
	B		3		6	7

Schedule	R				P
----------	---	--	--	--	---

Jobs					S	T
Machine	A				4	6
	B				6	7

Schedule	R			Q	P
----------	---	--	--	---	---

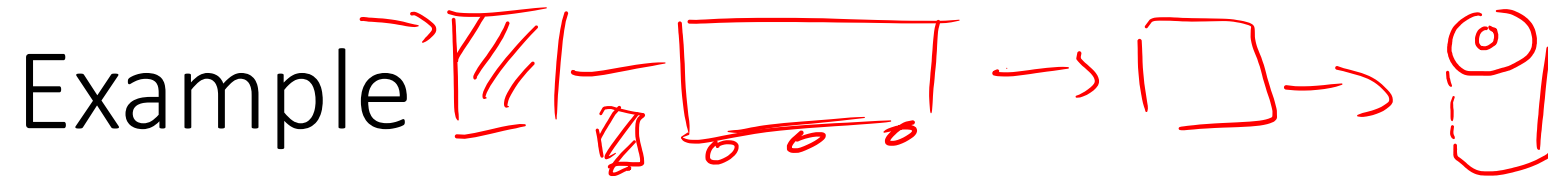
Jobs						T
Machine	A					6
	B					7

Schedule	R	S		Q	P
----------	---	---	--	---	---

Jobs						
Machine	A					
	B					

Schedule	R	S	T	Q	P
----------	---	---	---	---	---

*unique
optimal*



Jobs		P	Q	R	S	T
Machine	A	5	8	1	4	6
	B	2	3	11	6	7

Supply chain management

Schedule	R	S	T	Q	P
----------	---	---	---	---	---

After applying Johnson's Rule

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Machine	A																														
	B																														

	R
	S
	T
	Q
	P

Solution	Makespan	Idle Time of A	Idle Time of B	Total Idle Time
Original order	40	16	11	27
Johnson's	30	6	1	7

Example 2

Jobs		P	Q	R
Machine	A	2	4	6
	B	3	3	7

Schedule			
----------	--	--	--

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Machine	A																			
	B																			

	P
	Q
	R

Solution	Makespan	Idle Time of A	Idle Time of B	Total Idle Time
Original order	19	7	6	13

Example 2

argmin $i=1, j=1$ $\begin{bmatrix} 2 & 4 & 6 \\ 3 & 3 & 7 \end{bmatrix}$

CODE

EL

2

Jobs		P	Q	R
Machine	A	2	4	6
	B	3	3	7

Schedule			
----------	--	--	--

Jobs			Q	R
Machine	A		4	6
	B		3	7

Schedule	P		
----------	---	--	--

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Machine	A	Blue	Blue	Green	Green	Green	Green	Green	Green	Orange	Orange	Orange	Orange						
	B			Blue	Blue	Blue				Green	Green	Green	Green	Green	Green	Orange	Orange	Orange	Orange

Blue	P
Orange	Q
Green	R

Jobs				R
Machine	A			6
	B			7

Schedule	P		Q
----------	---	--	---

Jobs				
Machine	A			
	B			

Schedule	P	R	Q
----------	---	---	---

optimal

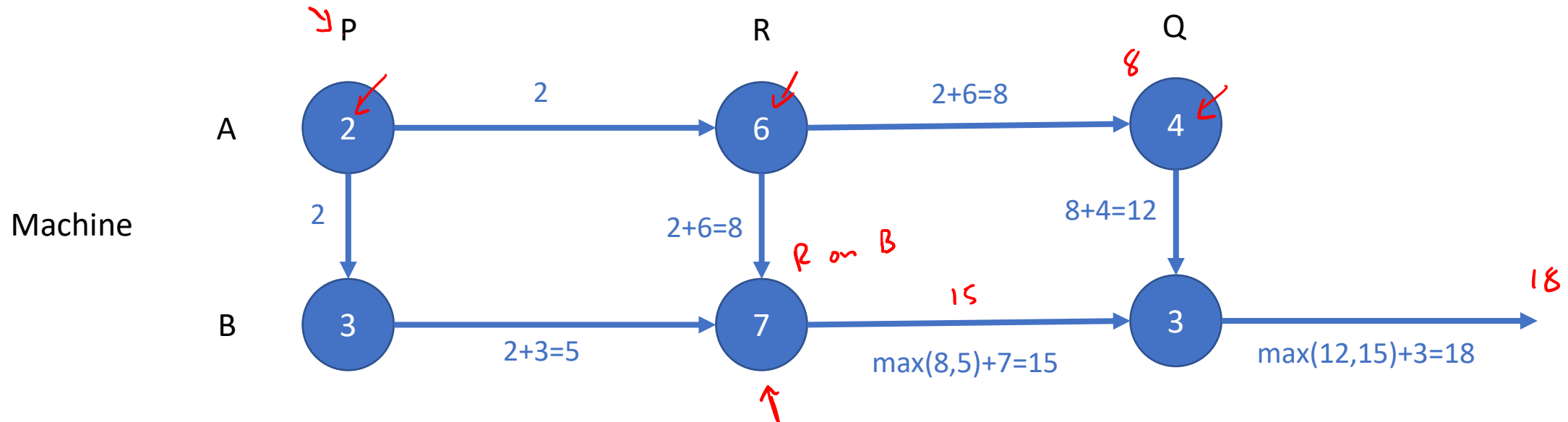
Solution	Makespan	Idle Time of A	Idle Time of B	Total Idle Time
Original order	19	7	6	13
Johnson's	18	6	5	11

Determining Makespan via Network

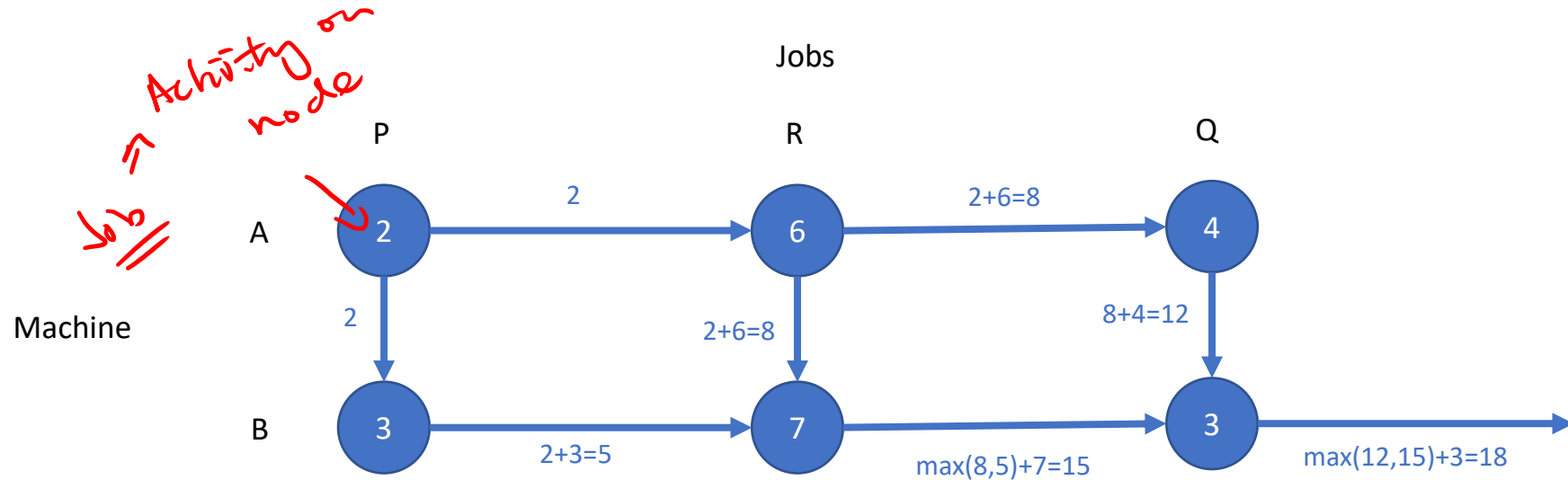
Jobs		P	Q	R
Machine	A	2	4	6
	B	3	3	7

Schedule	P	R	Q
----------	---	---	---

P R Q
↑
Jobs



Easily Converted to Excel



=G2+H2

	A	B	C	D	E	F	G	H	I	J	K
1											
2		P	R	Q		2	2	6	8	4	18
3		2	6	4		2		8		12	
4		3	7	3		3	5	7	15	3	
5											

Activity times copied over

=F3+F4

=MAX(G4,H3)+ H4

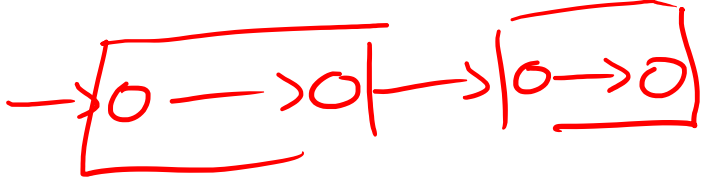
More Than 2 Machines

[3], 4, 5, -- .

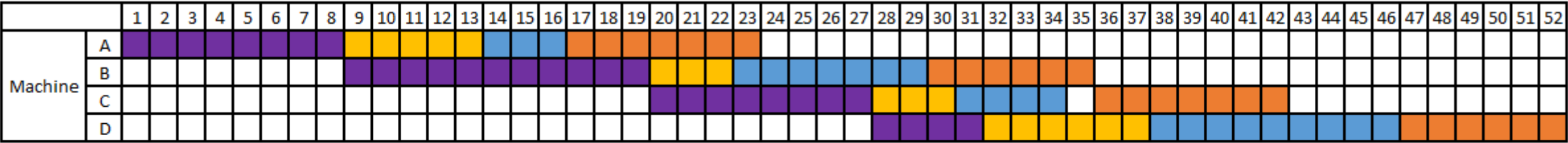
AB

CD

Jobs		P	Q	R	S
Machine	A	8	5	3	7
	B	11	3	7	6
	C	8	3	4	7
	D	4	6	9	6



Schedule				
----------	--	--	--	--



	P
	Q
	R
	S

Solution	Makespan	Idle Time of A	Idle Time of B	Idle Time of C	Idle Time of D	Total Idle Time
Original order	52	29	25	30	27	111

↑↑

↑

More Than 2 Machines

- Johnson's rule[✓] can be used to reduce the size of the search space.
 - Idea - reduce to a 2 stage problem. This is a heuristic! *Good*
 - Only produces a good solution, non necessarily optimal!
 - Often named the method/approach of Campbell Dudek and Smith *CDS*
- e.g. First and last stage considered

Jobs		P	Q	R	S
Machine	A	8	5	3	7
	B	11	3	7	6
	C	8	3	4	7
	D	4	6	9	6
Schedule					



Jobs		P	Q	R	S
Machine	A	8	5	3	7
	D	4	6	9	6
Schedule		R	Q	S	P

Makespan - 46

Other Possibilities

Aggregate the first two machines and last two machines

Jobs		P	Q	R	S
Machine	A	8	5	3	7
	B	11	3	7	6
	C	8	3	4	7
	D	4	6	9	6

Schedule				
----------	--	--	--	--



2 machines

Jobs		P	Q	R	S
Machine	A+B	19 ✓	8 ✓	10 ✓	13 ✓
	C+D	12 ✓	9 ✓	13 ✓	13 ✓

Schedule	Q	R	S	P
----------	---	---	---	---

Makespan - 46 ✓

Q R S P
R Q S P

Aggregate the first three machines and last three machines

Jobs		P	Q	R	S
Machine	A	8	5	3	7
	B	11	3	7	6
	C	8	3	4	7
	D	4	6	9	6

Schedule				
----------	--	--	--	--



Jobs		P	Q	R	S
Machine	A+B+C	27	11	14	20
	B+C+D	23	12	20	19

Schedule	Q	R	P	S
----------	---	---	---	---

Makespan - 48 ✓

For m machines

$m=4$

machine

1
2
3
4

1
2

1 + 2
3 + 4

1 + 2 + 3
2 + 3 + 4

$m - 1$ possible sequences

100

99

$$\begin{aligned} s_1 &= 1, \\ s_2 &= 1 + 2, \\ s_3 &= 1 + 2 + 3, \\ &\vdots \end{aligned}$$

$$s_{m-1} = 1 + 2 + \dots (m - 1), \quad 2 + 3 + \dots + m$$

1 + 2 + ... + 99

2 + 3 + ... + 100

4

m

$$(m - 1) + m$$

$$(m - 2) + (m - 1) + m$$

Palmer

Consider all these sequences and pick the best one!

$m!$

$(m - 1)$

Binary

Variables